

I-COR: Instruction-Level Fault Tolerance for Register File in 3-Stage Pipeline RISC-V Processors

ZEWEN CAO, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

YING YANG, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China

QI WANG, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China

The RISC-V architecture is increasingly deployed in safety-critical domains, such as automotive systems, yet the register file remains susceptible to Single Event Upsets (SEUs). Traditional fault-tolerance schemes like Triple Modular Redundancy (TMR) and Error Correction Codes (ECC) incur excessive area overhead ($>200\%$) or significant performance penalties (15.3–20.5% timing degradation), and critically, cannot prevent error accumulation. To address this, we propose **I-COR**, an instruction-level correction scheme that targets register file data errors. When an instruction accesses a corrupted register, I-COR replaces it with a corrective instruction pair: the first instruction rectifies the erroneous value through the write-back path to prevent error accumulation, while the second re-executes the original operation. By reusing existing forwarding paths, this pair maintains pipeline efficiency, introducing just one-cycle correction latency. Implemented on the Ibex core and synthesized in 28 nm, 110 nm CMOS, and 180 nm SOI technologies, I-COR reduces area overhead by 50.92–62.83% compared to TMR while avoiding ECC's timing degradation. Our automated two-stage fault-injection validation demonstrates I-COR's effectiveness: it achieves complete elimination of critical faults (0% occurrence for both application output mismatches and system hangs) and reduces non-critical faults (internal architectural errors without system-level impact) down to 0.12–3.17%, outperforming existing schemes by 2.6–28.8 $\times$. By combining minimal area, zero timing penalty, single-cycle correction latency, and guaranteed prevention of error accumulation, I-COR provides an efficient architectural-level fault-tolerance solution specifically designed for register file protection in safety-critical three-stage pipeline RISC-V processors.

CCS Concepts: • **Computer systems organization** → *Reliability; Processors and memory architectures*; • **Hardware** → *Error detection and error correction; Redundancy*; **System-level fault tolerance**;

Additional Key Words and Phrases: Fault tolerance, register file, RISC-V, error correction code (ECC), fault injection

ACM Reference Format:

Zewen Cao, Ying Yang, and Qi Wang. 2026. I-COR: Instruction-Level Fault Tolerance for Register File in 3-Stage Pipeline RISC-V Processors. *ACM Trans. Embedd. Comput. Syst.* 25, 2, Article 30 (March 2026), 25 pages. <https://doi.org/10.1145/3787103>

Authors' Contact Information: Zewen Cao, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China; e-mail: caozewen@ime.ac.cn; Ying Yang (corresponding author), Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China; e-mail: yangying@ime.ac.cn; Qi Wang, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China; e-mail: wangqi1@ime.ac.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1539-9087/2026/03-ART30

<https://doi.org/10.1145/3787103>

1 Introduction

The RISC-V architecture has seen exponential growth in recent years, with a **compound annual growth rate (CAGR)** of nearly 160% over the past five years. Its modular and open-source nature has made it a strong competitor to established architectures like ARM and x86, particularly in the automotive sector, where its adoption is projected to grow at a 217.7% CAGR. Many embedded applications traditionally relied on three-stage pipeline ARM processors such as the Cortex-M0, M1, M3, and M4 [3]. Today, three-stage pipeline RISC-V processors are increasingly adopted as direct alternatives, offering comparable pipeline efficiency. However, when deployed in automotive and other high-reliability environments, these processors face a critical challenge: **Single Event Upsets (SEUs)** can corrupt architectural states, leading to system failures—and in safety-critical systems, potentially catastrophic outcomes [8, 13].

Ensuring fault tolerance in these processors is therefore essential, particularly for register files, which are highly vulnerable to SEUs due to their role in storing critical operands. Traditional fault tolerance techniques, such as **Triple Modular Redundancy (TMR)** [28] and **Error Correction Codes (ECC)** [1, 7], face inherent tradeoffs. Although TMR offers robust error masking, its >200% area overhead is prohibitive for cost-sensitive designs. ECC improves area efficiency but introduces additional high latency [22, 25]. More critically, both methods inherently allow for the accumulation of errors in register files as they passively rely on write operations to refresh data. As a result, for infrequently accessed registers (e.g., RISC-V's GP register or ARM's R9 [2]), single-bit errors left uncorrected can accumulate into uncorrectable multibit faults.

Recent advancements have attempted to address the limitations of traditional fault-tolerant methods [5, 16, 17, 19, 20, 24, 29], but they often introduce new limitations. For example, [24] employs **dual modular redundancy (DMR)** to protect the register file in an FPGA-based soft core, achieving single-bit error masking with lower resource overhead than TMR, but it necessitates additional falling edge-triggered flip-flops. Ref. [5] introduces a low-complexity mechanism to anticipate data fetch and error correction in the L1 data cache (DL1), saving execution time compared to methods like “Extra cache cycles” and “Extra stage”. However, it is tailored for DL1 and requires two additional read ports and an extra adder, making it unsuitable for register files. Ref. [19] leveraged sensor data's natural redundancy to enhance ECC decoding, achieving 25% better error recovery. However, this data-dependent approach is not suitable for register files. Ref. [16] proposes a cache-based fault-tolerant protection method for the register file, but it increases **look-up tables (LUTs)** by 76%, and when considering borrowed cache resources, the overall hardware overhead becomes even more significant. Ref. [17] developed an enhanced ECC capable of correcting 1–4-bit burst errors while detecting 3-bit random errors, significantly improving coverage over conventional SEC-DED codes. However, this comes at the cost of 18% additional parity bits and fails to address error accumulation in rarely accessed registers. Ref. [29] proposed the HPR-Mul approximate redundancy technique, reducing TMR's area overhead by 60% while maintaining fault coverage through precision scaling. While effective for error-tolerant applications, its inherent approximation errors make it unsuitable for mission-critical operations requiring bit-exact correctness. Ref. [20] introduced a two-tiered ECC for **high-bandwidth memory (HBM)** that predicts and corrects **silent wordline decay (SWD)** errors proactively. Although this hierarchical approach reduces latency by 32%, it requires additional prediction logic, resulting in higher design complexity, making it unsuitable for register files. In conclusion, none provides a balanced solution that simultaneously minimizes area overhead, avoids performance penalties, and prevents error accumulation—a combination essential for automotive and other high-reliability applications.

To address these challenges, we propose an instruction-level correction (**I-COR**) scheme that converts SEU-affected instructions into corrective instruction pairs. Implemented on the Ibex core [18] alongside reference TMR and **single-error correction, double-error detection (SECDED)**

Hamming ECC [15] designs, I-COR uniquely leverages the existing write-back datapath to achieve timely error correction (adding just one clock cycle latency), thereby guaranteeing zero-error accumulation in register files. Our synthesis results under 28 nm CMOS, 110 nm CMOS, and 180 nm SOI technologies demonstrate I-COR's superior tradeoffs: (1) 50.92–62.83% area reduction versus TMR at comparable latency; (2) elimination of ECC's characteristic 15.3–20.5% timing degradation at comparable area costs. Furthermore, to validate I-COR's fault tolerance against error accumulation, we developed an automated two-stage fault injection system (see Section 4). Experimental results, categorized based on fault criticality (defined in Section 5.2), demonstrate complete elimination of critical faults (0% occurrence) while reducing non-critical faults down to 0.12–3.17% — representing a 2.6–28.8× improvement over conventional protection schemes (TMR scheme and ECC scheme).

This combination of area efficiency, timing preservation, and error-accumulation prevention makes I-COR particularly well suited for protecting register files in safety-critical three-stage pipeline RISC-V processors. Specifically, I-COR targets a practical functional-safety design space in which fault coverage must be carefully balanced against hardware overhead and execution latency. In this space, hardware-intensive schemes such as TMR are typically reserved for systems requiring the highest safety integrity level (ASIL-D), whereas lighter-weight protection mechanisms—such as ECC- or DMR-based designs—are more commonly adopted for ASIL-B or ASIL-C systems under stringent area and performance constraints. I-COR is explicitly designed for this latter class of systems. While ASIL-D-level fault tolerance would require complementary system-level mechanisms, this aligns with established ECC- and DMR-based practices. I-COR therefore provides an efficient register-file protection primitive with enhanced fault coverage (in terms of preventing error accumulation) and minimal timing impact.

The key contributions of this article are as follows:

- (1) **Instruction-level correction (I-COR)**: First scheme to transform SEU-affected instructions into corrective pairs, enabling low-overhead register file fault tolerance.
- (2) **Zero-error-accumulation guarantee**: Leverage the existing write-back data path to correct register file errors promptly, completely avoiding error accumulation.
- (3) **Two-stage fault injection system**: To rigorously validate I-COR's zero-error-accumulation property, we developed a command-driven, simulation-based framework that overcomes the limitations of conventional single-bit fault injection (e.g., TMR/ECC verification).
- (4) **Multi-node experimental validation**: Unlike most architecture-level works validated on a single node, this study comprehensively evaluates I-COR across 28 nm, 110 nm CMOS, and 180 nm SOI nodes, demonstrating its consistent advantages in area efficiency and timing preservation compared to conventional approaches.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 presents the proposed I-COR scheme. Section 4 describes the fault injection methodology. Section 5 reports evaluation results, Section 6 discusses extensibility and future directions, and Section 7 concludes.

2 Related Works

Traditional SEU protection methods for register files mainly include hardware-centric schemes such as TMR and ECC. To reasonably evaluate the proposed I-COR scheme, we implemented both TMR and ECC on our evaluation platform as baseline designs for quantitative comparison. Beyond these classic hardware-centric techniques, there also exist software-involved methods. To clarify the meaning of "instruction-level" in the I-COR scheme, we review representative **Software-Implemented Hardware Fault Tolerance (SIHFT)** techniques, which modify instruction flows to provide fault tolerance but typically incur considerable performance overhead. Finally, we

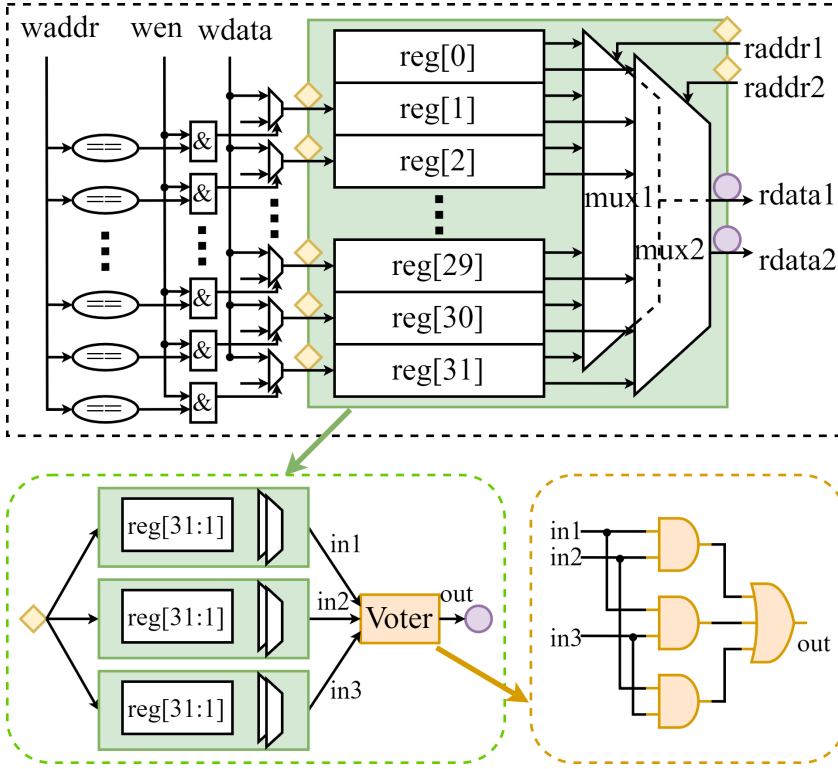


Fig. 1. Simplified diagram of the TMR scheme for the register file. The green dashed box (bottom-left) shows the simplified TMR structure, where input signals are triplicated and output signals are determined by majority voting. The yellow dashed box (bottom-right) highlights the voter circuit that selects the majority of three inputs as the final output.

also review works that aim to prevent errors from silently accumulating over time, such as the LiVe scheme, which ensures timely error detection in core-level light lockstep architectures for safety-critical processors.

2.1 TMR Scheme

Figure 1 illustrates a simplified block diagram of the TMR scheme for register files. The portion at the top enclosed by the black dashed box represents the register file structure of the target platform, where `reg[31:0]` denotes storage units, each reg being a 32-bit register, with the remainder comprising read/write logic. The portion at the bottom-left enclosed by the green dashed box depicts the simplified structure of TMR, with diamonds indicating TMR input signals and circles denoting TMR output signals. Input signals to redundant objects are triplicated, and the output signals from the redundant objects are determined by majority voting. The redundant objects primarily include storage units (`reg[31:1]`) and two selectors (`mux1`, `mux2`). It is noteworthy that according to the RISC-V specification, register `x0` is always 0 [6]; hence, `reg[0]` requires no redundancy. The portion at the bottom-right enclosed by the yellow dashed box outlines the circuit logic of the voter in TMR, which selects the majority of the three inputs as the final output.

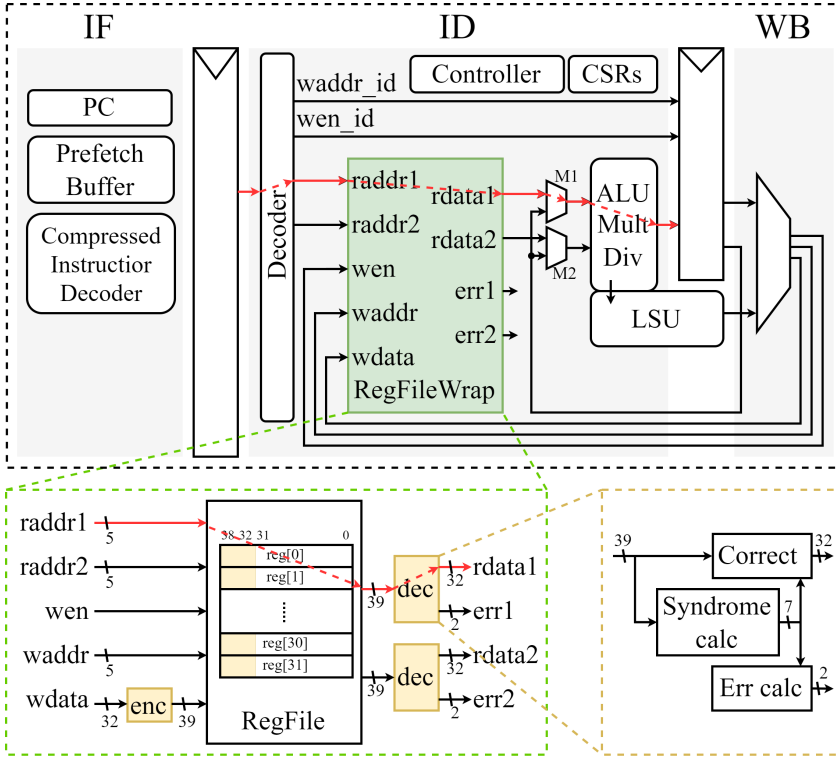


Fig. 2. Simplified diagram of the ECC scheme for the register file. The green dashed box (bottom-left) shows the internal structure of RegFileWrap, with ECC encoding (enc) and decoding (dec) modules added, and the register width extended to 39 bits (32 information + 7 redundancy). The yellow dashed box (bottom-right) highlights the decoding module, where error correction and error flag generation occur. The red arrows indicate the critical path of the processor core.

2.2 ECC Scheme

Figure 2 illustrates a simplified block diagram of the ECC scheme for the register file. The portion at the top enclosed by the black dashed box represents the three-stage pipeline structure of the Ibex core on the target platform, with RegFileWrap indicating the register file integrated with ECC encoding and decoding modules. In contrast to the unprotected register file, the ECC scheme adjusts only the register file section, leaving other Ibex structures unchanged. The portion at the bottom-left enclosed by the green dashed box details the internal structure of RegFileWrap, highlighting two main adjustments compared to the original register file:

- (1) Addition of an ECC encoding module (enc) and two ECC decoding modules (dec);
- (2) The register width increases from the original 32 bits to 39 bits, with the lower 32 bits serving as information bits and the upper 7 bits as redundancy bits, using SECDED Hamming codes as ECC.

The enc has simple logic and introduces a relatively small delay; thus, its path typically does not become a critical path in processor core design. However, the dec is relatively complex and introduces significant delay. The portion at the bottom-right enclosed by the yellow dashed box outlines the simplified internal structure of the dec. Data read from the register file undergoes

syndrome calculation to determine the syndrome, which is then used to correct the 32 information bits accordingly. The corrected `rdata` is sent to other modules of the processor. Additionally, `ErrCalc` generates two error flags based on the syndrome: `err[0]` indicates a single-bit error, which is correctable, while `err[1]` indicates a double-bit error, which is uncorrectable. These flags are routed to the top layer of the processor core, where higher-level systems can handle errors according to application requirements through software. In this article, only single-bit error protection is studied, without considering double-bit error protection. Considering that single-bit errors are the most common occurrences [30], this focus is acceptable.

2.3 SIHFT-Based Techniques

SIHFT-based techniques protect register files by modifying software instruction flows [4, 9–11]. Gonçalves et al. [11] proposed a low-level software-based fault-tolerance approach detecting SEUs in GPU register files through instruction duplication and consistency checks, effectively reducing **silent data corruption (SDC)** and hang errors but incurring up to $1.78\times$ runtime overhead (matrix multiplication), which may be unacceptable for real-time applications. Azambuja et al. [4] developed a non-intrusive hardware–software co-design integrating instruction duplication, signature verification, and watchdog modules, achieving full SEU and **single-event transient (SET)** coverage on MIPS processors with $1.33\text{--}1.53\times$ execution time and doubled memory usage. Gonçalves' subsequent work [10] introduced criticality-aware selective hardening, protecting only the top 30% vulnerable registers (ranked by SDC/**detected unrecoverable error (DUE)** **architectural vulnerability factor (AVF)**), reaching 90% fault coverage while hardening 54%/50%/48% of registers for Hotspot/Needleman–Wunsch/Quicksort, though DUE detection remained limited ($\leq 66\%$). The **Instruction Set Architecture (ISA)** extension approach [9] added three atomic instructions (`raLoad`, `raStore`, `raSetP`) with software redundancy, achieving 95.4%/100% SDC/DUE detection and halving runtime overhead, while relying on a customized ISA less compatible with general software–hardware ecosystems.

Unlike the software-level instruction concept in SIHFT, "instruction-level" in the I-COR scheme refers to a hardware-level concept, specifically implemented through processor microarchitecture design to protect register files. The I-COR solution is entirely transparent to software developers, fully compatible with the existing RISC-V software ecosystem, and requires no special conventions or custom instructions [9]. Specifically, when an instruction accesses a corrupted register, I-COR replaces it with a corrective instruction pair: the first instruction rectifies the erroneous value through the write-back path to prevent error accumulation, while the second re-executes the original operation. By reusing existing forwarding paths, this pair maintains pipeline efficiency, introducing just one cycle correction latency. Compared to SIHFT-centric solutions, this approach does not increase the space overhead of storage instructions. Moreover, in the absence of errors, its execution cycle count matches that of an unprotected architecture, and when errors are detected, it introduces only one additional cycle, making it friendly to real-time application scenarios. The design was inspired by the "vector instruction splitting" technique observed in the Xiangshan (Kunming Lake) open-source RISC-V processor project [21], where it is used to decompose complex instructions into multiple simpler micro-ops to facilitate out-of-order scheduling and parallelism optimization. Through our involvement with the Xiangshan project, we recognized its potential for fault-tolerance applications, leading to this novel adaptation for register file protection.

2.4 Error Accumulation Prevention Approaches

Beyond their area and latency overheads, conventional schemes such as TMR and ECC cannot prevent error accumulation in the register file. In particular, the ECC commonly used in processor designs is SECDED, which can detect up to two-bit errors but can correct only a single-bit error.

Since correction relies solely on write operations to refresh correct values, a single-bit error may persist in infrequently updated registers (e.g., RISC-V GP, ARM R9 [16]). If another upset occurs in a different bit of the same register before the first error is corrected, the accumulated multi-bit error exceeds SECDED's correction capability, resulting in an error that can be detected but not corrected, potentially leading to system-level failures. This accumulation effect is particularly critical in safety-sensitive domains such as automotive electronics.

To address this, another line of work has focused on bounding the lifetime of latent faults in processor state. Rather than correcting errors after they cause observable failures, these approaches aim to detect faults within a bounded latency to prevent them from remaining undetected and accumulating over time. A representative example is the LiVe scheme proposed by Hernandez and Abella [14]. LiVe builds on core-level light lockstep architectures, where two processor cores execute the same instruction stream in parallel. To reduce error detection latency, LiVe periodically exposes the architectural state (e.g., register contents) of both cores on the shared communication bus for cross-core comparison. This mechanism ensures that transient faults are detected within a bounded time, which is essential for meeting the safety requirements of automotive and other safety-critical systems.

While such lockstep schemes can effectively mitigate error accumulation, they depend on dual redundant cores and primarily provide detection, with recovery relying on costly mechanisms such as reset or rollback. In contrast, the I-COR scheme proposed in this work targets the register file of a three-stage in-order pipeline, offering localized, low-cost error detection and on-the-fly correction of transient upsets by leveraging lightweight information redundancy rather than full hardware replication. Permanent and common-cause faults, however, remain the responsibility of higher-level redundancy mechanisms. The two approaches, therefore, provide complementary solutions in different design tradeoff spaces.

3 Proposed Protection Scheme

The I-COR scheme differs from the TMR scheme or ECC scheme; besides adjustments to the register file section, modifications are also required for the processor core microarchitecture. Therefore, we will first introduce the processor core microarchitecture and the motivation for I-COR, followed by a discussion of the I-COR scheme for the register file. In this article, we selected the Ibex core as an illustration, which is an open-source three-stage pipeline RISC-V processor core having gained widespread attentions and practical applications.

3.1 Ibex Microarchitecture and the Motivation for I-COR

Ibex is a production-quality, open-source 32-bit RISC-V **Central Processing Unit (CPU)** core implemented in SystemVerilog. It is highly parameterized. In this article, it is configured as a three-stage pipeline scalar processor core that supports the RV32IMC instruction set extension, as depicted by the portion at the top enclosed by the black dashed box in Figure 2. The IF stage fetches instructions from memory based on the **program counter (PC)**. The ID stage decodes the fetched instructions into operations that the processor can execute, controlling **arithmetic logic units (ALUs)** and other functional units to perform computations as needed. The WB stage handles writing the results of executed instructions back to the register file.

The logic associated with the register file is concentrated in the ID and WB stages. Using the example of the instruction `add rd, rs1, rs2`, the following illustrates the read and write logic of the register file in the Ibex core.

The register file (corresponding to `RegFileWrap` in Figure 2) receives `rs1` and `rs2` through the `raddr1` and `raddr2` ports, which are the register indices corresponding to the source operands obtained from instruction decoding. It then asynchronously reads the source operands and outputs

them to the `rdata1` and `rdata2` ports. Selectors `M1` and `M2` choose between reading data from the register file and forwarding data from the WB stage based on data hazard detection results. The selected data is sent to the ALU for computation. The results from the ALU, along with the destination register index `waddr_id` and write enable signal `wen_id` from decoding, are sent to the ID-WB pipeline registers. Finally, these signals are selected by the selectors in the WB stage and sent to the `wen`, `waddr`, and `wdata` ports of the register file.

It is important to note that the propagation delay from the IF-ID pipeline registers to the ID-WB pipeline registers includes delays from modules such as the decoder, register file, ALU, and others. These path delays (see Figure 2, red arrow) are already considerable. Inserting large delay modules like `dec` (the yellow rectangle along the red-arrow path in Figure 2) in the register file could easily turn these paths into critical paths of the Ibex core. Therefore, careful consideration of timing is crucial in the fault-tolerant design of the register file.

Furthermore, the condition for a register file fault to cause a core failure is that the functional unit uses erroneous data read from the register file. By analyzing the microarchitectural implementation of the RV32IMC instruction set on Ibex, we observe that all instructions, including both single-cycle decode-execute instructions (e.g., `add`, `sub`, `and`, `addi`) and multi-cycle decode-execute instructions (e.g., `mul`, `div`), can read the register file exclusively only in the first cycle of the ID stage. This key insight reveals that protecting the register file is functionally equivalent to protecting instructions in the first cycle of the ID stage. Focusing on this first-cycle protection of the ID stage unifies the approach across instructions with varying ID-stage latencies, avoiding the complexity of designing separate protection schemes for different instruction types. This insight lays the foundation for the proposed I-COR scheme.

3.2 I-COR Scheme

The core idea of the I-COR scheme involves protecting the register file by transforming an instruction that encounters a single-bit error into a pair of instructions: the first instruction rectifies the erroneous value through the write-back path to prevent error accumulation, while the second re-executes the original operation. By reusing existing forwarding paths, this pair maintains pipeline efficiency, introducing just one cycle correction latency. To avoid the impact of SECEDED codes on subsequent evaluations, the I-COR scheme employs the same SECEDED codes as the ECC scheme. The I-COR scheme requires adjustments not only to the logic within the register file but also to the pipeline microarchitecture of the Ibex core, as depicted in Figure 3. We will now detail the protection scheme for the register file from three perspectives.

(1) Adjustment of the register file. Unlike the ECC scheme for the register file, the I-COR scheme necessitates gathering additional information from `RegFileWrap` to support adjustments in the Ibex microarchitecture. As shown in the bottom-left portion enclosed by the green dashed rectangle in Figure 3, apart from extracting corrected data (`sdata1`, `sdata2`) (shown in purple data flow), it also extracts information bits (`rdata1_ecc`, `rdata2_ecc`) (shown in green data flow) from the uncorrected data (`rdata1`, `rdata2`). The bottom-right portion enclosed by the yellow dashed rectangle in Figure 3 illustrates an ECC correction example. The writing data `wdata` is `32'h0010_0001`, which is encoded by `enc` to `39'h59_0010_0001` as `wdata_ecc`. Then, `wdata_ecc` is written into `reg[2]`. A SEU flips the least significant bit of `reg[2]` from 1 to 0, causing the read result `rdata1_ecc` to change to `39'h59_0010_0000`. The information bit of `rdata1_ecc` is `rdata1`, which is `32'h0010_0000` and different from the write data `wdata`, so it is considered incorrect data. The data `sdata1` corrected by `dec` is `32'h0010_0001`, which matches the write data `wdata`, making it the correct data. At the same time, `err1` is `2'b01`, indicating a single-bit error in `rdata1`.

(2) Adjustment of the Ibex microarchitecture data path. The portion at the top enclosed by the black dashed box in Figure 3 shows the adjustment of the Ibex microarchitecture data path,

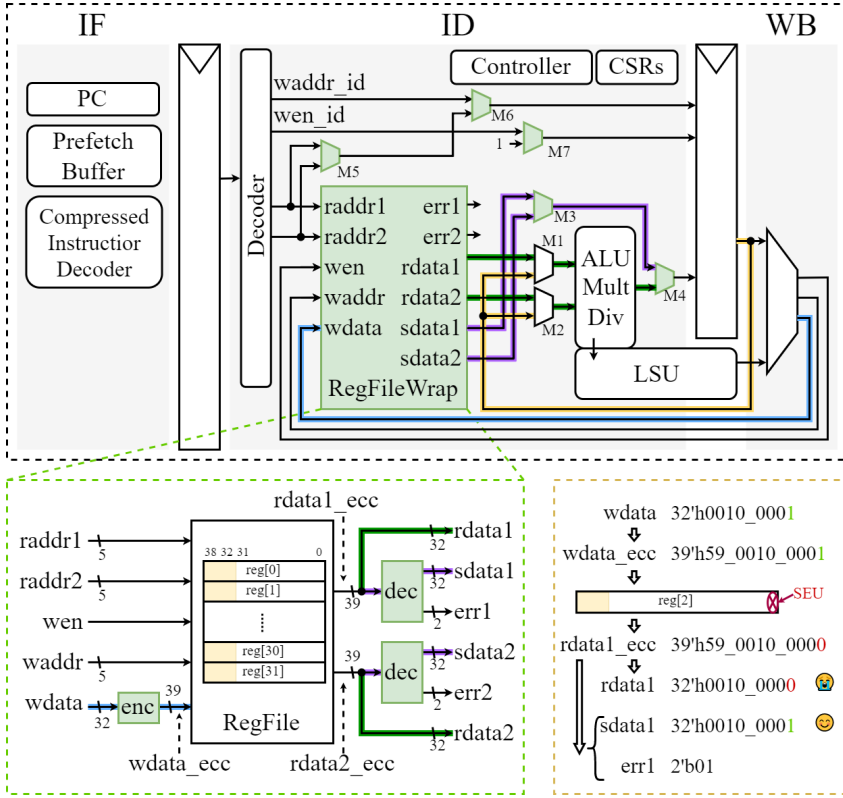


Fig. 3. Simplified diagram of the I-COR scheme for the register file in a three-stage pipeline RISC-V processor, highlighting four colored data flows: green (normal instruction path in ID stage), purple (correction instruction path in ID stage), blue (writeback path of corrected data in WB stage), and yellow (forwarding path from WB to ID stage).

featuring four distinct data flows marked by colored arrows. `rdata1` and `rdata2` of `RegFileWrap` are signals of uncorrected data, which are sent to ALUs and other functional units after being selected by selectors M1 and M2 to complete instruction functions (shown in green data flow, representing normal instructions in the ID stage). Their results are then sent to the ID-WB pipeline registers through selector M4. This is the data path for reading data from the register file when not affected by SEU, which increases only the delay of M4 compared to the unhardened path. The `sdata1` and `sdata2` of `RegFileWrap` are signals of corrected data, which are sent to the ID-WB pipeline registers through selectors M3 and M4 after being selected (shown in purple data flow, representing correction instructions in the ID stage). This is the data path for reading data from the register file when affected by SEU, which, despite including the delay of the decoding module `dec`, is parallel to the path containing functional units such as ALU (shown in green data flow). In general, the delay of the former path is less than that of the latter path. In the WB stage, corrected data is written back to the register file through the blue data flow, preventing single-bit errors in the register file from accumulating into multi-bit errors. This critical writeback path improves the reliability of the register file. Furthermore, the yellow data flow represents the forwarding path from WB to ID stage, which delivers corrected data to subsequent normal instructions following correction instructions. This forwarding mechanism ensures pipeline efficiency — specifically, it introduces only a 1-cycle

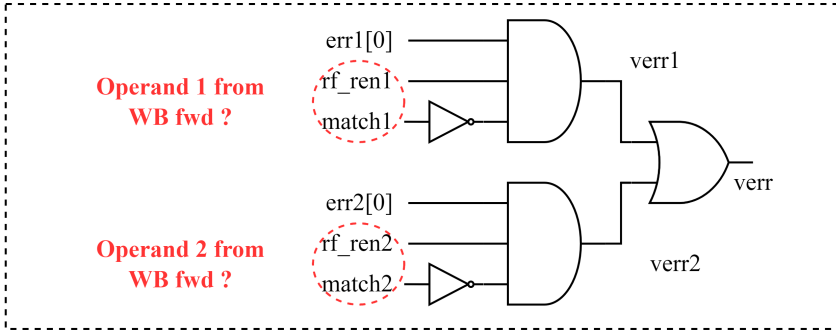


Fig. 4. Logic circuit diagram for generating valid single-bit error flags of the register file, illustrating the relationships among error signals ($err1$, $err2$), read-enable signals (rf_ren1 , rf_ren2), and address match conditions ($match1$, $match2$).

penalty upon error detection. Additionally, the register indexes $raddr1$ and $raddr2$, corresponding to the two decoded source operands, record the register indices that require correction. After being selected by selectors M5 and M6, they are sent to the ID-WB pipeline registers. The write enable signal 1 of the correction instruction is sent to the ID-WB pipeline registers after being selected by selector M7. During the WB stage, these signals guide the writeback path (shown in blue data flow) to write the corrected data back to the appropriate registers in the register file.

(3) Adjustment of the Ibex microarchitecture control path. Figure 4 shows the logic circuit diagram for generating valid single-bit error flags in `RegFileWrap`. Here, the $err1[0]$ and $err2[0]$ correspond respectively to the single-bit error flags for $err1$ and $err2$ signals of `RegFileWrap` in Figure 3. The rf_ren1 and rf_ren2 indicate whether the current valid instruction needs to read the register file. The $match1$ and $match2$ indicate whether the read register file address at the ID stage matches the write register file address at the WB stage. The $verr1$ and $verr2$ indicate whether valid single-bit errors have occurred in the registers indexed by $raddr1$ and $raddr2$. The $verr$ is obtained through the OR operation of $verr1$ and $verr2$. If $match1$ is 1, selector M1 in Figure 3 will select forwarding data from the WB stage. And $rdata1$ from `RegFileWrap` will not be used. Therefore, when $match1$ is 1, even if both $err1[0]$ and rf_ren1 are 1, $verr1$ will be 0, indicating that the single-bit error is considered an invalid error and will be ignored. The $verr2$ is like $verr1$, and the details are omitted. This is consistent with our description in Section 3.1: “the condition for a register file fault to cause a core failure is that the functional unit uses erroneous data read from the register file”. If the functional unit uses data forwarded from the WB stage instead, it will not trigger I-COR’s error correction mechanism. Moreover, when triggered, the forwarding logic from the WB stage can provide corrected operands for subsequent normal instructions in the following cycle. This means that regardless of the instruction type encountered, I-COR’s error correction mechanism introduces only one additional cycle of latency.

The signals $verr1$, $verr2$, and $verr$ are key signals for adjusting the control path of the Ibex microarchitecture. They achieve I-COR schemes by controlling the selection signals of the selectors within the top portion enclosed by the black dashed rectangle in Figure 3. When $verr1$ is 1, certain signals related to write register file operations passing into the ID-WB pipeline registers are affected: the register file write enable signal selects 1, $raddr1$ is selected as the write address signal, and $sdata1$ is chosen as the write data signal. Similarly, when $verr2$ is 1, these signals are affected: the register file write enable signal selects 1, $raddr2$ is selected as the write address signal, and $sdata2$ is chosen as the write data signal. On the other hand, when $verr$ is 0, the signals related to write register file operations passing into the ID-WB pipeline registers maintain their original

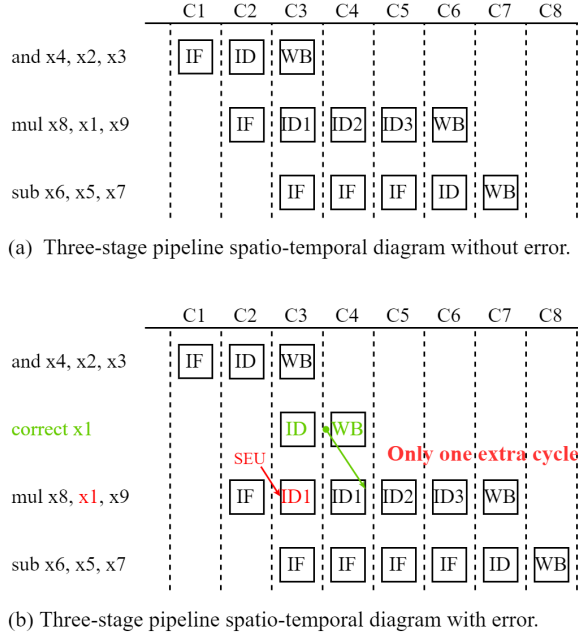


Fig. 5. Spatio-temporal diagrams of the three-stage pipeline for instructions and, mul, and sub: (a) execution without SEU errors; (b) execution with a single-bit error in register x1, where the mul instruction in the ID1 stage triggers I-COR's error correction mechanism.

logic: wen_id is selected as the register file write enable signal, waddr_id is selected as the write address signal, and the calculation results of the ALU and other functional units are selected as the write data signal. Additionally, in pipeline technology, to prevent the original instruction (affected by SEU) at the ID stage from being overwritten by instructions from the IF stage, when verr is 1, the ID stage needs to stall and ensure that the original instruction is not executed.

To better illustrate the I-COR scheme, we present here a concrete example that explicitly demonstrates when a register-file SEU is detected, how it is identified as a valid error during an ID-stage register read, and how this event subsequently triggers and completes the I-COR instruction-level correction process before normal execution resumes. Specifically, we consider the error correction process when a single-bit error occurs in the instruction mul in the ID stage as an example. Figure 5(a) depicts the spatio-temporal diagram of the three-stage pipeline for instructions and, mul, and sub when no SEU errors are present. Since the execution of the instruction mul in the Ibex processor takes 3 clock cycles, we mark the ID stages of instruction mul as ID1, ID2, and ID3. And it is during ID1 that the instruction mul performs a read operation on the register file. There are no RAW data hazards [26] between adjacent instructions; thus, selectors M1 and M2 in the ID stage choose rdata1 and rdata2 from RegFileWrap as operands. Simultaneously, this implies that if dec detects a single-bit error of the register file in ID1, verr will generate a valid single-bit error flag and trigger I-COR's error correction mechanism. Figure 5(b) illustrates the spatio-temporal diagram of the three-stage pipeline for instructions and, mul, and sub when a SEU occurs in register x1. After two clock cycles C1 and C2, in clock cycle C3, the instruction and reaches the WB stage, the instruction mul reaches the ID stage, and the instruction sub reaches the IF stage. Because register x1 has experienced a SEU and the instruction mul in the ID stage reads register x1, dec detects a valid single-bit error, setting verr1 to 1 and triggering I-COR's error correction mechanism:

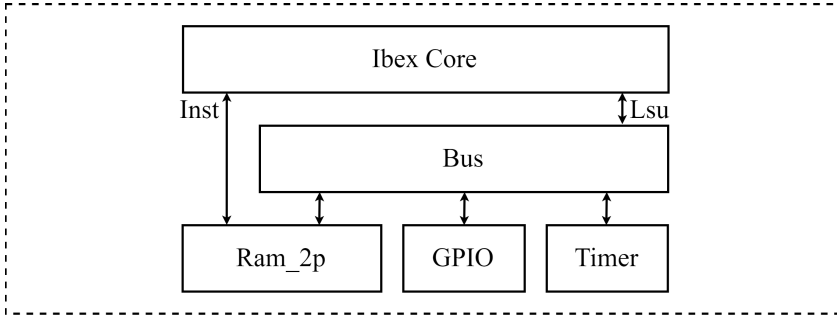


Fig. 6. Simplified block diagram of the Ibex SoC, showing the instruction bus directly connected to Ram_2p, and the data bus connected via Bus to Ram_2p, GPIO, and Timer.

- (1) Stall the execution of the instruction mul in the ID stage.
- (2) Insert the logic of correction instructions correct x1 in the ID stage and pass the signals, which are related to write register file operations of correct x1, to the ID-WB pipeline registers.

Thus, in clock cycle C4, the instruction correct x1 reaches the WB stage, and the corrected data is written into the register file. In the same clock cycle (C4), the instruction mul is re-executed in the ID stage. Since the write address of the register file for correct x1 from the WB stage matches the read address of the register file for the instruction mul in the ID stage (match1 = 1, as shown in Figure 4), selector M1 in Figure 3 chooses the forwarded corrected data from the WB stage as operands for the ALU. This ensures the correct functionality of the instruction mul when re-executed at the ID stage. Additionally, since match1 is 1, verr is set to 0. This causes the control path to exit I-COR's error correction mechanism, ensuring that subsequent instructions are executed correctly. In clock cycles C5 and C6, the instruction flow resumes normal pipeline execution. It can be observed that the proposed I-COR scheme introduces only one clock cycle during correction. Moreover, this correction is transparent to the software, ensuring seamless compatibility with the RISC-V software ecosystem. Additionally, upon detecting a valid single-bit error, correction of the erroneous register in the register file can immediately occur in the next cycle, significantly reducing the probability of multiple single-bit errors accumulating into multi-bit errors.

4 Fault Injection Method

Section 3 introduced I-COR's architecture and theoretical fault-tolerance mechanism. However, to empirically validate its resilience, fault injection experiments are essential. Fault injection methods include hardware-based, software-based, and simulation-based approaches. Among these, simulation-based fault injection is widely adopted during the system design phase due to its advantages in controllability, observability, and cost-effectiveness. While traditional fault tolerance schemes like TMR and ECC can effectively handle single-bit faults, their fundamental inability to prevent error accumulation renders conventional single-bit fault injection methods inadequate for verifying I-COR's zero-error-accumulation guarantee. To overcome this limitation, we developed an advanced two-stage RTL-level Verilog HDL-based simulation system using command-driven techniques.

Figure 6 presents a simplified diagram of the Ibex **system-on-chip (SoC)**. The instruction bus interface of the Ibex core is directly connected to Ram_2p, while the memory access bus interface of the Ibex core is connected via Bus to Ram_2p, GPIO, and Timer. Ram_2p supports dual-read and

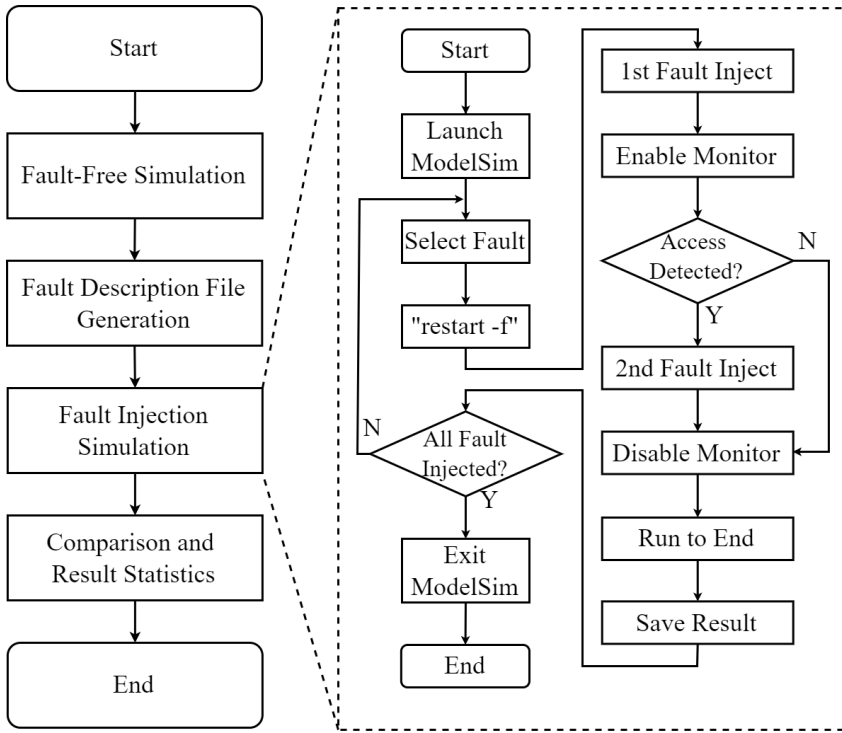


Fig. 7. Automated workflow of the two-stage fault injection simulation system, showing the sequence from fault-free simulation to fault description generation, Tcl-controlled fault injection in ModelSim, and final result comparison with Python scripts.

single-write ports. The instruction bus interface of the Ibex core is restricted to read operations only. Upon reset, the Ibex core's PC points to Ram_2p, and driven by the clock, the Ibex core sequentially executes instructions fetched from Ram_2p.

Figure 7 illustrates the workflow in the two-stage fault injection simulation system.

- (1) Initially, a fault-free simulation is executed to obtain the architecture state of interest, which serves as the reference result.
- (2) Subsequently, a Python script extracts the reference names of registers in the target injection modules from the VCD waveform file and generates a fault description file randomly. This file encompasses multiple faults, each characterized by a triplet (*f_{itm}*, *f_{ipa}*, *f_{imask}*). Here, *f_{itm}* denotes the fault injection time, *f_{ipa}* specifies the index of the register file, and *f_{imask}* represents the mask of the register bits designated for flipping.
- (3) Following this, the fault injection simulation is executed under the control of a custom-developed Tcl script. This script processes fault line-by-line from the fault description file. First, at each predetermined *f_{itm}* point, it simulates a SEU by inverting register bits specified by the *f_{ipa}* and *f_{imask}* parameters using the "change" command. Next, a monitor is enabled to detect when the same register (addressed by *f_{itm}*) is effectively accessed — either when the register's data is consumed by a functional unit or when it is overwritten by new data. Then, upon detection, the system triggers another SEU (by flipping a different bit in the same register) and subsequently disables the monitor. Finally, the simulation continues until the current iteration concludes. The process repeats iteratively until all fault injections are completed,

Table 1. The Comparisons of Implementation Resource and Timing

		Unprotected	TMR	ECC*	I-COR
Timing (MHz)	28nm CMOS	659	659	558↓	659
	110nm CMOS	126	126	103↓	126
	180nm SOI	39	39	31↓	39
Area (μm^2)	28nm CMOS	12194.5	21564.4	15305.3	15269.9
	110nm CMOS	158459.1	265740.6	185610.6	185041.8
	180nm SOI	283100.8	494112.3	315644.0	316243.3

* The ECC scheme exhibited timing violations under the given timing constraints, so this column presents its synthesis results at the maximum clock frequency.

with results automatically logged. Notably, the implementation of a “restart -f” command enables batch execution of multiple fault injections following a single launch of ModelSim (see the right side of Figure 7). This flow yields substantial time savings compared to conventional approaches requiring separate simulator initialization for each fault injection [11].

- (4) Finally, another Python script is used to compare the results of the fault injection simulation with those of the fault-free simulation, categorizing statistics according to specific domain requirements. This comprehensive simulation process, in conjunction with the Makefile script, establishes a fully automated workflow, substantially enhancing simulation efficiency.

5 Evaluation

To rigorously evaluate the I-COR scheme, we implemented the unprotected scheme, TMR scheme, ECC scheme, and I-COR scheme on the same target platform, Ibex SoC (refer to Figure 6). We conducted area and timing assessments under three different technologies, in conjunction with evaluating their protection effectiveness using the same fault injection method (refer to Figure 7).

5.1 Area and Timing Evaluation

Three protection schemes were implemented in SystemVerilog and integrated into the Ibex SoC platform. The Ram_2p memory was set to 4 MB. The ECC scheme and the I-COR scheme use SECDED Hamming codes as ECC. Modifying the defined macro parameters allows for control over which scheme is synthesized into the RTL design. The Ibex Core was synthesized for four distinct configurations across three technology nodes, utilizing Synopsys Design Compiler.

Table 1 compares the timing and area of the unprotected scheme (Unprotected), TMR scheme (TMR), ECC scheme (ECC), and I-COR scheme (I-COR). In terms of timing, the synthesis results under 28 nm CMOS technology show that, under a timing constraint of 659 MHz, neither the unprotected scheme, TMR scheme, nor I-COR scheme exhibited timing violations. However, the ECC scheme did show a timing violation. Additionally, the first few paths with the highest slack values all pass through the dec module in Figure 2, which is consistent with our analysis in Section 2.2, indicating that the delay introduced by the ECC decoding module is relatively large. This makes the ECC scheme more likely to limit the processor core’s clock frequency. After multiple synthesis iterations with a 1 MHz step increment, we determined that the maximum clock frequency for the ECC scheme is 558 MHz, as shown in the ECC column of Table 1. This indicates that the I-COR scheme avoids 15.3% of the timing degradation compared to the ECC scheme under 28nm CMOS technology. Correspondingly, the I-COR scheme avoids 18.3% and 20.5% of the timing degradation compared to the ECC scheme under the 110 nm CMOS process and 180 nm SOI technology, respectively. This demonstrates that the I-COR scheme has good optimization effects in different technologies and performs even better under slower technologies.

Table 2. The Comparisons of Area Increments

		$\Delta \text{Area}(\mu\text{m}^2)^a$	% of ΔArea^b
28nm CMOS	TMR	9369.9	76.84%
	ECC	3110.8	25.51%
	I-COR	3075.4	25.22%
110nm CMOS	TMR	107281.5	67.70%
	ECC	27151.5	17.13%
	I-COR	26582.7	16.78%
180nm SOI	TMR	211011.5	74.54%
	ECC	32543.2	11.50%
	I-COR	33142.5	11.71%

$$^a \Delta \text{Area}_{\text{scheme}} = \text{Area}_{\text{scheme}} - \text{Area}_{\text{unprotected}}$$

$$^b \% \text{ of } \Delta \text{Area}_{\text{scheme}} = \frac{\Delta \text{Area}_{\text{scheme}}}{\text{Area}_{\text{unprotected}}} \times 100\%.$$

In terms of area, it can be observed that the area of the TMR scheme is the largest under all three technologies. For each technology, the area overhead of each scheme is calculated as a percentage increase relative to the unprotected baseline implementation. The results are summarized in Table 2. Under the 28 nm CMOS, 110 nm CMOS, and 180 nm SOI technologies, the I-COR scheme reduces the area increase by 51.62%, 50.92%, and 62.83%, respectively, compared to the TMR scheme. The area increase is comparable to that of the ECC scheme. In the cases of the 28 nm CMOS and 110 nm CMOS technologies, the area increase of the I-COR scheme is even less than that of the ECC scheme. This reduction is attributed to the optimization of critical paths. The optimization reduced the number of buffers inserted for timing enhancement. In summary, our scheme achieves significant timing optimization with minimal area overhead.

5.2 Protection Effectiveness Evaluation

In this study, we utilized the fault injection method described in Section 4 to inject faults into the unprotected scheme, TMR scheme, ECC scheme, and I-COR scheme. All fault-injection experiments were conducted at the RTL-level, using cycle-accurate Verilog HDL-based simulation. The fault injection workloads included 5 benchmark tests from `riscv-tests` [27]. Their parameter `DATA_SIZE` was configured to appropriate sizes to ensure that these workloads all executed for comparable durations.

- (1) **Median**: This is a median filtering algorithm with a window size of 3. It smooths the signal by replacing each data point with the median value within its neighborhood. In this study, the input data size, `DATA_SIZE`, is configured as 800. The workload runs for 21,679 cycles in the simulation environment.
- (2) **Memcpy**: This is a function used for copying data in memory. It serves as a benchmarking tool to help developers and system administrators understand and optimize the memory and processor performance of the system. In this study, the quantity of data to be copied, `DATA_SIZE`, is configured as 800. The workload runs for 18,436 cycles in the simulation environment.
- (3) **Multiply**: This is a software implementation of multiplication. It achieves multiplication operations through simple operations such as addition and shifting. In this study, the quantity of data for multiplication, `DATA_SIZE`, is configured as 50 sets. The workload runs for 22,291 cycles in the simulation environment.
- (4) **Rsort**: This is a counting sort algorithm suitable for sorting integers within a certain range. In this study, the number of array elements to be sorted, `DATA_SIZE`, is configured as 64. The workload runs for 21,564 cycles in the simulation environment.

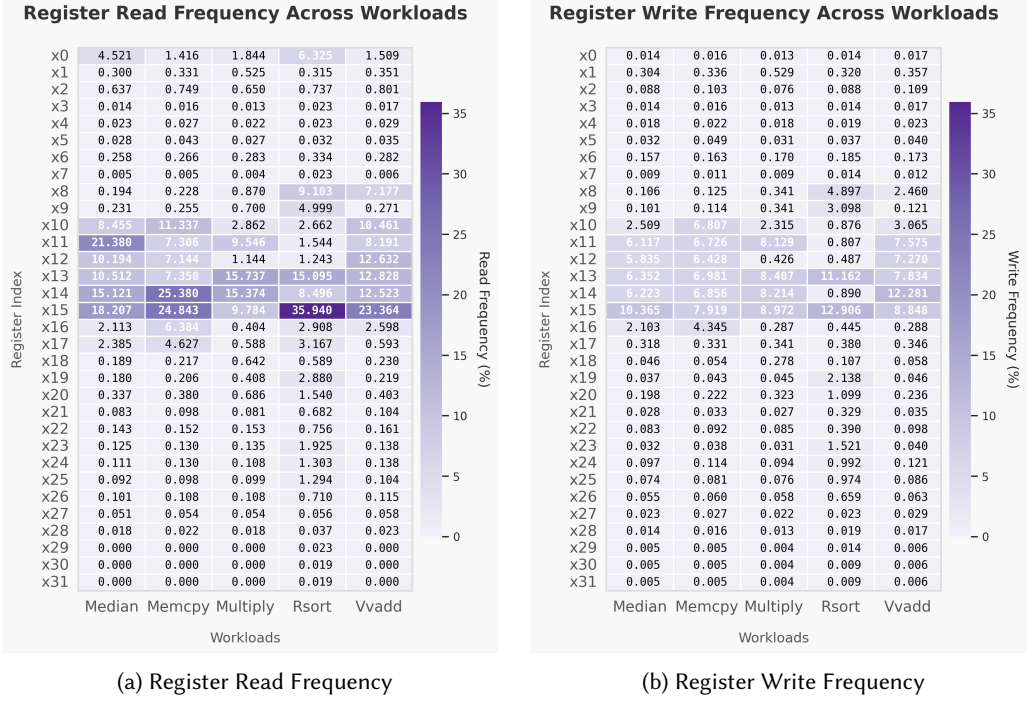


Fig. 8. Register access frequency heatmaps across five workloads: (a) read operations and (b) write operations. Color intensity indicates access frequency (darker = higher).

- (5) **Vvadd**: This is a test program for vector addition. It involves adding corresponding elements of two vectors and storing the result in a new vector. In this study, the number of elements in the vectors, `DATA_SIZE`, is configured as 800. The workload runs for 17,360 cycles in the simulation environment.

Figure 8 presents heatmaps of register file access frequencies across five workloads (*Median*, *Memcpy*, *Multiply*, *Rsort*, and *Vvadd*), with read and write operations visualized separately in Figure 8(a) and (b). The access frequency is calculated as follows:

$$Freq_{op}^r(W) = \left(\frac{T_{op}^r(W)}{T_{run}(W)} \right) \times 100\% \quad (1)$$

where $T_{op}^r(W)$ denotes cycles when register $r \in [x0, x31]$ undergoes operation $op \in \{\text{read, write}\}$ during workload W 's execution, and $T_{run}(W)$ represents W 's total execution cycles. All 32 registers were accessed, with registers x10–x15 demonstrating the highest utilization. Notably, registers x27–x31 showed minimal utilization ($<0.1\%$) and were only written to in *Median*, *Memcpy*, *Multiply*, and *Vvadd* workloads. *Rsort* displayed distinct behavior with significantly higher read frequencies, particularly in register x15.

Prior to evaluating the protection schemes, a clear fault impact classification was established based on prior work [23]. The outcomes are categorized into four types based on system-level consequences:

- (1) **Correct Result**: All architectural states (PC, registers) and application outputs remain correct.

- (2) **Non-critical fault (Architecture Internal Failure, AIF)**: The test results and PC are correct, but the register file is incorrect. Note: AIF represents a non-critical fault as system-level functionality remains intact. Such transient architectural mismatches can be overwritten by subsequent operations or corrected by protection mechanisms.
- (3) **Critical fault (Application Output Mismatch, AOM)**: The test results are incorrect. Note: AOM represents a critical fault, as it results in erroneous system-level functionality outputs. For example, it may lead to failures in automotive braking systems.
- (4) **Critical fault (Hang)**: The workload fails to complete execution within the specified time. Note: Hang also represents a critical fault, as it indicates an erroneous execution flow at the system level. For example, it may lead to deviations in the vehicle's trajectory.

To quantitatively assess the system vulnerability, we introduce the AVF, defined as the probability that an injected fault will manifest as either an AOM or Hang — the two critical failure modes identified in our classification. The AVF is computed as

$$AVF = \frac{N_{AOM} + N_{Hang}}{N_{Inject}} \quad (2)$$

where N_{Inject} denotes the total number of effectively injected faults (excluding those that failed to trigger secondary fault injection), while N_{AOM} and N_{Hang} count the occurrences of each critical failure mode, respectively. This metric serves as a key indicator for comparing the effectiveness of different protection schemes in our subsequent analysis.

To quantify the statistical reliability of the AVF estimates, we further report their 95% **confidence intervals (CIs)**. These are calculated using a binomial distribution approximation, where each fault injection trial is modeled as a Bernoulli experiment (critical vs. non-critical outcome). The error margin is derived as

$$CI = \hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{N_{Inject}}} \quad (3)$$

where $\hat{p}$ denotes the observed AVF. This ensures that the results in Table 3 not only provide average system vulnerability but also convey statistically rigorous error bounds.

To adhere to the principle of controlling variables in comparative experiments, the same fault description file (containing 10000 faults) was used when simulating fault injection in four different scenarios under the same workload. After excluding faults that did not trigger secondary fault injection, the number of valid faults for each of the five workloads was: 7,893 (*Median*), 8,237 (*Memcpy*), 7,663 (*Multiply*), 8,721 (*Rsort*), and 7,949 (*Vvadd*). Table 3 presents a comprehensive analysis of these experiments, showing the percentage distributions of four distinct outcome types — Hang, AOM, AIF, and Correct — observed during two-stage fault injection testing across all five workloads. The table further provides the calculated AVF estimates according to Equation 2, along with their corresponding 95% CIs for each of the four schemes evaluated. The three key attributes characterizing our fault injection methodology and the detailed two-stage injection workflow were previously established in Section 4 and are not repeated here to avoid redundancy.

The fault injection results demonstrate compelling evidence for the effectiveness of various protection schemes against different error types. As shown in Table 3, the unprotected configuration exhibits expected high failure rates, with Hang errors reaching 43.66% in *Rsort* and AOM errors peaking at 24.84% in the same workload, reflecting the vulnerability of unprotected systems. Both the TMR scheme and our proposed I-COR scheme completely eliminate these critical faults (0.00% occurrence), while the ECC scheme shows residual vulnerabilities, particularly in register-intensive workloads like *Rsort* (25.89% Hang and 5.33% AOM). This observation aligns with the high register

Table 3. Error Type Distribution and AVF Results with 95% Confidence Intervals for Four Protection Schemes Across Five Benchmarks

Workload	Scheme	Correct	AIF	AOM	Hang	AVF	95% CI of AVF
<i>Median</i>	Unprotected	17.95%	38.59%	18.60%	24.86%	43.46%	[42.36%, 44.55%]
	TMR	94.92%	5.08%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
	ECC	80.93%	6.13%	2.84%	10.10%	12.94%	[12.20%, 13.68%]
	I-COR	99.78%	0.22%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
<i>Memcpy</i>	Unprotected	16.69%	39.21%	17.02%	27.07%	44.09%	[43.02%, 45.17%]
	TMR	91.82%	8.18%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
	ECC	80.38%	9.13%	1.17%	9.32%	10.49%	[9.83%, 11.15%]
	I-COR	96.83%	3.17%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
<i>Multiply</i>	Unprotected	28.41%	26.73%	11.37%	33.50%	44.86%	[43.75%, 45.98%]
	TMR	99.19%	0.81%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
	ECC	76.61%	3.38%	4.10%	15.91%	20.01%	[19.11%, 20.90%]
	I-COR	99.88%	0.12%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
<i>Rsort</i>	Unprotected	22.14%	9.36%	24.84%	43.66%	68.50%	[67.53%, 69.48%]
	TMR	89.24%	10.76%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
	ECC	60.92%	7.85%	5.33%	25.89%	31.22%	[30.25%, 32.20%]
	I-COR	98.68%	1.32%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
<i>Vvadd</i>	Unprotected	19.11%	35.51%	19.68%	25.70%	45.38%	[44.28%, 46.47%]
	TMR	94.88%	5.12%	0.00%	0.00%	0.00%	[0.00%, 0.00%]
	ECC	81.09%	5.93%	4.05%	8.93%	12.98%	[12.24%, 13.72%]
	I-COR	99.70%	0.30%	0.00%	0.00%	0.00%	[0.00%, 0.00%]

Note: The number of valid faults is 7,893 (*Median*), 8,237 (*Memcpy*), 7,663 (*Multiply*), 8,721 (*Rsort*), and 7,949 (*Vvadd*).

read frequencies shown in Figure 8(a) (x15: 35.940%, x13: 15.095%, x8: 9.103%, x14: 8.496%). The higher the register read frequency, the more likely errors in the register file propagate to processor outputs, ultimately causing AOM or Hang failures.

In terms of non-critical fault handling, the results reveal significant differences between protection approaches. For AIF, TMR scheme demonstrates variable performance with rates ranging from 0.81% (*Multiply*) to 10.76% (*Rsort*), while ECC scheme maintains a more consistent but still substantial AIF presence (3.38–9.13%). Notably, I-COR achieves exceptional suppression of AIF across all workloads, limiting it to 0.12–3.17%, representing an order-of-magnitude improvement over conventional methods (specifically, a 2.6–23.6× improvement over the TMR scheme and 2.9–28.8× over the ECC scheme). This performance is particularly remarkable considering I-COR simultaneously maintains correct execution rates between 96.83–99.88%, outperforming both the TMR scheme (89.24–99.19%) and ECC scheme (60.92–81.09%) in fault tolerance effectiveness.

The workload-specific analysis reveals interesting patterns in fault resilience. Register-intensive operations like *Rsort* prove most challenging for all protection schemes, with the ECC scheme's correct execution rate dropping to 60.92% — its worst performance across all tests. In contrast, I-COR maintains 98.68% correct execution for this demanding workload, demonstrating particular effectiveness for high-frequency register read operations. The *Multiply* workload shows another notable pattern, where I-COR achieves 99.88% correct execution with merely 0.12% AIF, suggesting special effectiveness for computational operations. These consistent results across diverse workloads confirm I-COR's comprehensive protection capability.

The experimental data further reveal that conventional approaches exhibit significant workload-dependent variability. TMR scheme's correct execution rates fluctuate between 89.24–99.19%,

while ECC scheme shows an even wider variation (60.92–81.09%). In contrast, I-COR maintains consistently high reliability (all workloads >96% correct execution) with minimal performance degradation. I-COR's consistent performance across diverse workloads makes it an exceptionally robust solution for varied computing scenarios.

From an AVF perspective, both TMR and I-COR achieve perfect fault tolerance (0.00% AVF) across all workloads, but they differ fundamentally in their protection mechanisms and practical reliability. While TMR relies on hardware redundancy, our results show I-COR provides superior overall protection with significantly lower AIF rates (0.12–3.17% vs. TMR's 0.81–10.76%) and higher correct execution rates (96.83–99.88% vs. TMR's 89.24–99.19%). Importantly, the current fault injection tests only targeted a single module within TMR's triple redundancy. Simultaneous identical bit-flips in any two modules could bypass TMR's protection and lead to critical faults, while I-COR's architectural approach remains resilient against such correlated faults.

It is also important to note that the execution time overhead of I-COR in valid fault scenarios is deterministic and negligible. As analyzed in Section 3.2, I-COR introduces one additional cycle whenever a fault is detected. Therefore, valid fault injections involving two faults lead to either two extra cycles if both faults are detected (resulting in *Correct*) or one extra cycle if only one fault is detected (resulting in *AIF*). Since the delay is strictly bounded (at most two cycles) and independent of the workload, it has a negligible impact on the overall performance.

In summary, the experimental results demonstrate that I-COR provides comprehensive fault protection with three key characteristics:

- (1) Complete mitigation of critical faults: I-COR achieves 0.00% occurrence of Hang and AOM errors, matching TMR's perfect AVF while avoiding ECC's residual vulnerabilities (up to 31.22% critical faults in register-intensive workloads).
- (2) Effective control of non-critical faults: I-COR reduces AIF rates to 0.12–3.17%, significantly lower than TMR (0.81–10.76%) and ECC (3.38–9.13%).
- (3) Consistent performance across workloads: I-COR maintains correct execution rates above 96% in all tested scenarios, including register-intensive operations where ECC drops to 60.92% and TMR to 89.24%.

These results confirm that I-COR is suitable for safety-critical applications at ASIL-B and ASIL-C, and may support ASIL-D compliance when integrated with appropriate system-level fault-tolerance mechanisms.

Combined with the Area and Timing Evaluation, these results collectively demonstrate that I-COR represents an excellent solution that effectively balances area overhead, performance penalty avoidance, and fault tolerance capabilities.

5.3 Comparison with Existing Protection Scheme

While Section 5.1 and Section 5.2 quantitatively evaluated I-COR alongside implemented baselines (TMR and ECC), this subsection provides a holistic comparison with existing protection schemes. We first summarize the main quantitative findings regarding TMR and ECC, then present a qualitative comparison with SIHFT-based schemes and the LiVe scheme.

Summary of TMR and ECC comparison. TMR and ECC are two classic hardware-centric protection schemes widely used for soft error mitigation. Our synthesis results (Tables 1 and 2) show that TMR incurs the largest area overhead across all technology nodes, while I-COR achieves comparable area to ECC and reduces area increase by 50.92–62.83% compared to TMR. In terms of timing, ECC exhibits critical-path delays that degrade its maximum operating frequency by 15.3%–20.5% relative to I-COR. Fault injection experiments (Table 3) further reveal that while both TMR and I-COR completely eliminate critical faults (0.00% AVF), I-COR significantly reduces non-critical faults

Table 4. Qualitative Comparison between I-COR and Representative Existing Schemes

Aspect	SIHFT-based	LiVe	I-COR (This Work)
Target scope	ISA-visible state (mainly RF)	Core-level (lockstep)	RF
Implementation	SW / HW-SW co-design	HW (redundant cores)	HW (microarchitecture)
Typical overhead	1.3–1.8× time, 2× memory	≈2× area	≈0% time, low area overhead
Protection style	Detect + recovery (SW)	Detect (system-level recovery)	Detect + correct (local HW)
Recovery latency	Moderate (local SW re-exec)	Long (core reset / rollback)	1-cycle (local HW correction)
ISA compatibility	ISA ext. / recompile	Compatible	Full compatible
Real-time support	Low (high baseline delay)	Moderate (only on errors)	High (deterministic, low latency)

Abbreviations: RF = Register File; SW = Software; HW = Hardware.

(0.12%–3.17% AIF) compared to TMR (0.81%–10.76%), and achieves higher correct execution rates (96.83%–99.88% vs. 89.24%–99.19%). In contrast, ECC leaves substantial residual vulnerabilities, with AVF up to 31.22% in register-intensive workloads. Overall, I-COR achieves a balanced tradeoff between protection and cost, mitigating the high area overhead of TMR and the timing degradation and residual vulnerabilities of ECC.

Qualitative comparison with SIHFT-based schemes. Existing SIHFT methods protect ISA-visible state (e.g., register files) by modifying instruction flows, but face several well-known limitations. Instruction duplication methods (e.g., Gonçalves et al. [11]) require redundant computations, incurring up to 1.78× runtime overhead that challenges real-time system requirements. Hardware-software co-design approaches (e.g., Azambuja et al. [4]) rely on additional watchdog modules, effectively doubling memory usage. Selective hardening techniques (e.g., Gonçalves et al. [10]) trade coverage (90%) for efficiency, while ISA extension schemes (e.g., [9]) achieve 95.4% SDC detection rates but require custom instruction sets, creating compatibility issues with existing ecosystems. In contrast, the proposed I-COR scheme adopts a fundamentally different design paradigm. It leverages hardware-automated instruction pairs for on-the-fly correction: when accessing corrupted registers, the processor dynamically generates instruction pairs consisting of corrective operations followed by original instruction re-execution, reusing existing write-back paths to achieve single-cycle correction latency (see Section 3.2 for design details). I-COR introduces zero runtime overhead in error-free scenarios and only a fixed one-cycle penalty during correction, significantly outperforming SIHFT's 1.33–1.78× performance degradation. Moreover, I-COR maintains standard RISC-V binary compatibility without requiring any ISA modifications or software recompilation, while imposing no extra memory access pressure. Notably, its design is inspired by the vector instruction splitting technique of the Xiangshan processor [21], representing a novel cross-domain repurposing of a microarchitectural feature for fault tolerance.

Qualitative comparison with LiVe. LiVe focuses on reducing error detection latency in lockstep-based safety-critical processors, while I-COR targets low-cost fault tolerance specifically for register files. These two approaches are therefore complementary: LiVe optimizes error detection at the system level, whereas I-COR provides efficient correction at the module level. Lockstep designs (including LiVe) mainly detect errors, with recovery requiring hardware resets, software retries, or state rollbacks that incur substantial delays. In contrast, I-COR detects and corrects errors locally within one cycle, offering much lower recovery latency. We emphasize that LiVe represents a well-established and broadly applicable technique: by operating at the core level, it can protect a wide range of processor structures (pipelines, caches, control logic, etc.) without requiring detailed microarchitectural modifications. This generality makes LiVe particularly suitable for complex industrial processors, while our work intentionally specializes in register file protection for streamlined three-stage pipelines, achieving higher efficiency for this specific use case.

As summarized in Table 4, I-COR achieves efficient register file protection with near-zero overhead, deterministic real-time behavior, and full binary compatibility, addressing the limitations

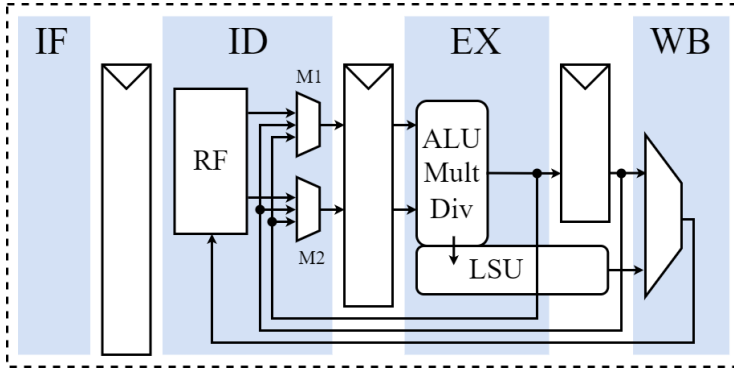


Fig. 9. Simplified four-stage RISC-V pipeline architecture, illustrating the split between the ID and EX stages and the placement of forwarding path selectors that source data from the EX and WB stages.

of SIHFT-based software techniques. Unlike LiVe, which focuses on rapid error detection and relies on costlier system-level recovery mechanisms (e.g., resets or rollbacks), I-COR performs both detection and correction locally within the register file, achieving much lower recovery latency. Meanwhile, LiVe remains advantageous in terms of generality and system-level coverage, highlighting the complementary nature of these two design paradigms.

It should be noted, however, that I-COR's current implementation is tailored for a three-stage pipeline and has inherent limitations in scaling to other pipeline architectures (see Section 6), which poses opportunities for future research.

6 Extensibility and Future Directions

This section discusses the extensibility of I-COR and outlines future research directions, with the clarification that I-COR is not intended as a standalone end-to-end safety solution for ASIL-D systems, but as a low-overhead microarchitectural building block that can be extended and complemented by system-level mechanisms.

Although the I-COR scheme is currently applied to a three-stage pipeline RISC-V processor, its core principle — instruction splitting — indicates potential extensibility to processors with deeper pipelines. The design is based on the following key principles:

- **ECC-based error correction** to resolve the area overhead of the TMR scheme.
- **Parallel execution of ECC decoding and functional units** to minimize latency.
- **Writing corrected data back to the register file** using subsequent pipeline stages to prevent error accumulation.
- **Forwarding of corrected data** to allow normal instructions to continue with only one additional cycle during correction, thereby maintaining overall performance.

A natural extension is to apply I-COR to processors with deeper pipelines, such as the CV32E40P [12], a representative four-stage RISC-V core featuring data forwarding. We therefore discuss a conceptual extension of the I-COR scheme to a four-stage pipeline (Figure 9), where the architecture is simplified to highlight key data flows. The four-stage pipeline splits the ID stage into separate ID and EX stages, building upon the three-stage pipeline (Figure 3). To balance delays, the design positions forwarding path selectors in the ID stage, allowing data to be sourced from either the EX or WB stage, similar to the bypass network implemented in processors like CV32E40P.

In this conceptual four-stage pipeline, the I-COR scheme could operate as follows:

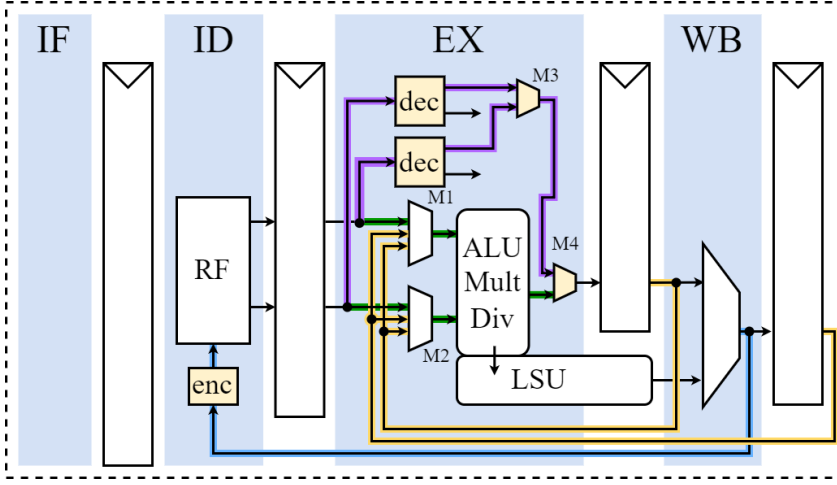


Fig. 10. Extension of the I-COR scheme to a four-stage RISC-V pipeline. The diagram highlights the parallel ECC decoding (purple) and functional unit execution (green), the corrected data written back to the register file (blue), and the forwarding of corrected data (yellow). Compared to the three-stage pipeline design, the forwarding selector is shifted from the ID stage to the EX stage, and additional forwarding paths from WB and post-WB stages to EX are introduced to ensure timely availability of corrected values.

- **ECC-based error correction** (see dec and enc in Figure 10) could address the TMR area overhead.
- **Parallel execution of ECC decoding and functional units** (see purple and green data flows in Figure 10) could reduce latency.
- **Corrected data** (see blue data flow in Figure 10) could be written back to the register file to prevent error accumulation.
- **Forwarding corrected data** (see yellow data flow in Figure 10) could allow normal instructions to proceed with minimal delay, maintaining processor performance.

For the four-stage pipeline, some adjustments might be required:

- Since dec is in the EX stage, errors would be detected, and instruction splitting would occur at EX.
- To enable normal instructions in EX to use the corrected data, additional forwarding paths from WB and post-WB stages to EX are needed (see the two yellow data flows in Figure 10). Since the data selected by the forwarding selector in EX is identical to and correct compared to the data selected by the ID stage's selector, the ID-stage selector becomes redundant once the EX-stage selector is introduced. Therefore, the ID-stage selector is removed in Figure 10. This is equivalent to shifting the forwarding logic one stage later in the pipeline, in order to meet the fault-tolerant requirements of the I-COR architecture.

For deeper pipelines and multi-core architectures, new challenges may arise. For example, subdividing arithmetic units into more pipeline stages could require further splitting of the dec module, complicating the adaptation of the I-COR scheme. These limitations highlight the need for more flexible solutions.

On the other hand, the I-COR scheme is designed to provide efficient detection and on-the-fly correction of transient faults at the microarchitectural level, significantly reducing hardware overhead compared to full TMR. It can also detect permanent faults and propagate error signals

(via a 1-bit error flag) to higher system layers, enabling timely recovery actions such as software retries, state rollbacks, or controlled system resets. However, it is important to clarify that I-COR does not provide real-time fault masking for permanent faults in the same manner as TMR; instead, it functions as a fast and lightweight detection-and-correction mechanism that enhances the responsiveness of system-level recovery strategies.

Accordingly, I-COR should be viewed not as a standalone end-to-end solution for automotive-grade safety (e.g., ISO 26262 ASIL-D), but rather as a microarchitectural building block that is primarily suited for ASIL-B and ASIL-C systems, or as a complementary component within higher-level ASIL-D solutions when integrated with appropriate system-level mechanisms. In this context, its low base overhead makes I-COR an attractive candidate to be combined with techniques such as time-staggered execution, diverse redundancy, or other system-level resilience approaches. Future work will explore such hybrid approaches, evaluating how I-COR can contribute to achieving comprehensive fault coverage while maintaining better cost-effectiveness than traditional DMR/TMR-based designs.

Future work will focus on two directions: (1) extending I-COR to deeper pipelines and multi-core systems, including resilience enhancements for register files; and (2) integrating I-COR with higher-level fault-tolerance techniques — such as time-staggered execution or diverse redundancy — to build efficient, comprehensive resilience solutions for safety-critical applications.

7 Conclusion

Traditional fault tolerance techniques, such as TMR and ECC, incur substantial area overhead ($>200\%$) and performance penalties (15.3–20.5% timing degradation) while remaining ineffective in mitigating error accumulation in infrequently accessed registers. To address these limitations, we analyze the microarchitectural implementation of the RV32IMC instruction set on Ibex and observe that protecting the RISC-V register file against SEUs is functionally equivalent to safeguarding instructions in the first cycle of the ID stage. Leveraging this insight, we propose I-COR, an instruction-level correction scheme that transforms SEU-affected instructions into corrective pairs, utilizing existing write-back paths to eliminate error accumulation with minimal overhead. Synthesis results across 28 nm, 110 nm CMOS, and 180 nm SOI technologies reveal that I-COR achieves superior tradeoffs compared to conventional methods. Specifically, it reduces area overhead by 50.92–62.83% relative to TMR while avoiding the timing degradation inherent to ECC. Furthermore, we have designed an automated two-stage fault injection system. Simulation results confirm I-COR's protection effectiveness, demonstrating complete elimination of critical faults (0%) and a reduction of non-critical faults to 0.12–3.17%, outperforming existing protection schemes by 2.6–28.8 $\times$. In summary, I-COR delivers a highly efficient, balanced solution that minimizes area overhead, preserves timing performance, and prevents error accumulation. Future work will further explore architectural-level fault-tolerant designs and extend I-COR's applicability to diverse microarchitectures.

References

- [1] Aurelien Alacchi, Edouard Giacomin, Scott Temple, Roman Gauchi, Michael Wirthlin, and Pierre-Emmanuel Gaillardon. 2023. Low latency SEU detection in FPGA CRAM with in-memory ECC checking. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 5 (May 2023), 2028–2036. DOI: <https://doi.org/10.1109/TCSI.2023.3243644>
- [2] ARM Limited. 2006. Procedure Call Standard for the ARM Architecture. Version 2.04, ARM Limited, Cambridge, UK.
- [3] ARM Ltd. n.d. Arm Processor Architectures. Retrieved January 8, 2026 from <https://developer.arm.com/Processors>
- [4] José Rodrigo Azambuja, Ângelo Lapolli, Lucas Rosa, and Fernanda Lima Kastensmidt. 2011. Detecting SEEs in micro-processors through a non-intrusive hybrid technique. *IEEE Transactions on Nuclear Science* 58, 3 (June 2011), 993–1000. DOI: <https://doi.org/10.1109/TNS.2011.2109398>

- [5] Pedro Benedicte, Carles Hernandez, Jaume Abella, and Francisco J. Cazorla. 2019. LAEC: Look-ahead error correction codes in embedded processors L1 data cache. In *Proceedings of the 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Florence, Italy, 818–823. DOI : <https://doi.org/10.23919/DATE.2019.8714877>
- [6] RISC-V Foundation. 2014. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.0*. Retrieved January 8, 2026 from <https://github.com/riscv/riscv-isa-manual>
- [7] Zhen Gao, Yinghao Cheng, Qiang Liu, Anees Ullah, and Pedro Reviriego. 2023. Efficient protection of FPGA implemented LDPC decoders against single event upsets (SEUs) on configuration memories. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 9 (Sept. 2023), 3770–3780. DOI : <https://doi.org/10.1109/TCSI.2023.3279444>
- [8] Roger Goerl, Paulo Villa, Fabian L. Vargas, César A. Marcon, Nilberto H. Medina, Nemitala Added, and Marcilei A. Guazzelli. 2019. Combined ionizing radiation & electromagnetic interference test procedure to achieve reliable integrated circuits. *Microelectronics Reliability* 100–101 (Sept. 2019), 113341. DOI : <https://doi.org/10.1016/j.microrel.2019.06.033>
- [9] M.M. Gonçalves, J.E. Rodriguez Condia, M. Sonza Reorda, L. Sterpone, and J.R. Azambuja. 2020. Improving GPU register file reliability with a comprehensive ISA extension. *Microelectronics Reliability* 114 (Nov. 2020), 113768. DOI : <https://doi.org/10.1016/j.microrel.2020.113768>
- [10] Marcio Goncalves, Fernando Fernandes, Ivan Lamb, Paolo Rech, and Jose Rodrigo Azambuja. 2019. Selective fault tolerance for register files of graphics processing units. *IEEE Transactions on Nuclear Science* 66, 7 (July 2019), 1449–1456. DOI : <https://doi.org/10.1109/TNS.2019.2903027>
- [11] Marcio Gonçalves, Mateus Saquetti, Fernanda Kastensmidt, and Jose Rodrigo Azambuja. 2017. A low-level software-based fault tolerance approach to detect SEUs in GPUs' register files. *Microelectronics Reliability* 76–77 (Sept. 2017), 665–669. DOI : <https://doi.org/10.1016/j.microrel.2017.07.035>
- [12] OpenHW Group. 2024. *CORE-V CV32E40P User Manual, Version 1.8.3*. Retrieved January 8, 2026 from <https://github.com/openhwgroup/cv32e40p>
- [13] Jon Gutiérrez-Zaballa, Koldo Basterretxea, and Javier Echanobe. 2024. Evaluating single event upsets in deep neural networks for semantic segmentation: An embedded system perspective. *Journal of Systems Architecture* 154 (Sept. 2024), 103242. DOI : <https://doi.org/10.1016/j.sysarc.2024.103242>
- [14] Carles Hernandez and Jaume Abella. 2014. LiVe: Timely error detection in light-lockstep safety critical systems. In *Proceedings of the 51st Annual Design Automation Conference*. ACM, San Francisco CA USA, 1–6. DOI : <https://doi.org/10.1145/2593069.2593155>
- [15] M. Y. Hsiao. 1970. A class of optimal minimum odd-weight-column SEC-DED codes. *IBM Journal of Research and Development* 14, 4 (1970), 395–401. DOI : <https://doi.org/10.1147/rd.144.0395>
- [16] Zongnan Liang, Jiawei Nian, Hongjin Liu, Xuru Wang, and Mengfei Yang. 2023. C-DMR: A cache-based fault-tolerant protection method for register file. *The Journal of Supercomputing* 79, 4 (Mar. 2023), 4383–4397. DOI : <https://doi.org/10.1007/s11227-022-04836-2>
- [17] He Liu, Liyi Xiao, Tianqi Wang, and Jie Li. 2024. Error detection and correction codes for memories with enhanced detection abilities. In *Proceedings of the 2024 IEEE 7th International Conference on Electronics and Communication Engineering (ICECE)*. IEEE, Xi'an, China, 165–168. DOI : <https://doi.org/10.1109/ICECE63871.2024.10976832>
- [18] lowRISC. n.d. *The Ibex Documentation*. Retrieved January 8, 2026 from <https://ibex-core.readthedocs.io/en/latest/>
- [19] Yair Mazal and Hugo Guterman. 2023. Enhanced error correction employing natural redundancy of sensor data. *IEEE Transactions on Communications* 71, 10 (Oct. 2023), 5675–5685. DOI : <https://doi.org/10.1109/TCOMM.2023.3298977>
- [20] Youngki Moon, Seung Ho Shin, Seokjun Jang, Duyeon Won, and Sungho Kang. 2025. A novel prediction-based two-tiered ECC for mitigating SWD errors in HBM. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 2 (Feb. 2025), 488–498. DOI : <https://doi.org/10.1109/TVLSI.2024.3474791>
- [21] OpenXiangShan. 2021. DecodeUnitComp.scala in XiangShan RISC-V Processor. Retrieved September 2, 2025 from <https://github.com/OpenXiangShan/XiangShan/blob/master/src/main/scala/xiangshan/backend/decode/DecodeUnitComp.scala>
- [22] Soumitra Pal, Sayonee Mohapatra, Wing-Hung Ki, and Aminul Islam. 2021. Soft-error-aware read-decoupled SRAM with multi-node recovery for aerospace applications. *IEEE Transactions on Circuits and Systems II: Express Briefs* 68, 10 (Oct. 2021), 3336–3340. DOI : <https://doi.org/10.1109/TCSII.2021.3073947>
- [23] Alexis Ramos, Ricardo G. Toral, Pedro Reviriego, and Juan Antonio Maestro. 2019. An ALU protection methodology for soft processors on SRAM-based FPGAs. *IEEE Transactions on Computers* 68, 9 (Sept. 2019), 1404–1410. DOI : <https://doi.org/10.1109/TC.2019.2907238>
- [24] Alexis Ramos, Anees Ullah, Pedro Reviriego, and Juan Antonio Maestro. 2018. Efficient protection of the register file in soft-processors implemented on xilinx FPGAs. *IEEE Transactions on Computers* 67, 2 (Feb. 2018), 299–304. DOI : <https://doi.org/10.1109/TC.2017.2737996>
- [25] Matteo Sonza Reorda, Luca Sterpone, and Anees Ullah. 2017. An error-detection and self-repairing method for dynamically and partially reconfigurable systems. *IEEE Transactions on Computers* 66, 6 (June 2017), 1022–1033. DOI : <https://doi.org/10.1109/TC.2016.2607749>

- [26] A. Shrivastava, E. Earlie, N.D. Dutt, and A. Nicolau. 2006. Retargetable pipeline hazard detection for partially bypassed processors. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 14, 8 (Aug. 2006), 791–801. DOI : <https://doi.org/10.1109/TVLSI.2006.878468>
- [27] RISC-V Software. n.d. riscv-tests. Retrieved January 8, 2026 from <https://github.com/riscv-software-src/riscv-tests.git>
- [28] Chiyu Tan, Yan Li, Xu Cheng, Jun Han, and Xiaoyang Zeng. 2021. General efficient TMR for combinational circuit hardening against soft errors and improved multi-objective optimization framework. *IEEE Transactions on Circuits and Systems I: Regular Papers* 68, 7 (July 2021), 3044–3057. DOI : <https://doi.org/10.1109/TCSI.2021.3076185>
- [29] Jafar Vafaei and Omid Akbari. 2024. HPR-Mul: An area and energy-efficient high-precision redundancy multiplier by approximate computing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 11 (Nov. 2024), 2012–2022. DOI : <https://doi.org/10.1109/TVLSI.2024.3445108>
- [30] R. Velazco, P. Cheynet, and R. Ecoffet. 1999. Effects of radiation on digital architectures: One year results from a satellite experiment. In *Proceedings of the 12th Symposium on Integrated Circuits and Systems Design (Cat. No.PR00387)*. IEEE Comput. Soc, Natal, Brazil, 164–169. DOI : <https://doi.org/10.1109/SBCCI.1999.803112>

Received 19 June 2025; revised 18 December 2025; accepted 23 December 2025